



Clingonia

A GUI for managing and visualizing
Flatland environments.

Adrian Henze and Max Vogel

Final Report
by MadMotion

Railway Scheduling in Flatland
University of Potsdam
Germany
March 2025

Table of Contents

1	Introduction.....	4
1.1	Motivation	4
1.2	Structure	5
1.3	Technical Requirements	5
2	Implementation	6
2.1	main.py	6
2.2	build_gif.py	6
2.3	build_png.py	6
2.4	clingo_actions.py	7
2.5	config.py	7
2.6	custom_canvas.py	7
2.7	custom_widgets.py	8
2.8	files.py	9
2.9	gen_png.py	9
2.10	load_env.py	9
2.11	positions.py	10
2.12	views.py	10
3	Project Results	12
3.1	Menus & Generation.....	12
3.2	Build & Simulation	13
4	Discussion	14
5	Future Research	15

Abstract. This report introduces Clingonia, a revolutionary technology transforming railway scheduling in Flatland. More than just a graphical user interface, Clingonia is an intelligent command center that simplifies managing, visualizing, and solving Flatland environments with unprecedented ease.

Harnessing the raw power of Clingo and Answer Set Programming, Clingonia turns complex work steps into an intuitive experience. Generate, build, save, load and optimize environments seamlessly, all in one unified interface. Dive into multiple visualization modes: numbered paths for prompt results, detailed timetables for clarity, and animated GIFs for dynamic insights, and watch your solutions come to life.

Whether you are an expert or a newcomer, Clingonia redefines what is possible with Flatland environments.

1 Introduction

1.1 Motivation

Before the introduction of Clingonia, the workflow lacked structure, efficiency, and a centralized system to handle the various tasks required for designing, visualizing, and solving railway problems with ASP. In the absence of a unified platform, we were forced to rely on external tools, scattered software solutions, or even traditional methods such as pen and paper to conceptualize and design railway environments. This approach was not only slow but also prone to errors and inconsistencies.

One of the hardest parts of this process was the manual construction of custom environments. Each environment had to be coded from scratch, cell by cell, requiring precise assignment of coordinates and track identifiers. This task became exponentially more difficult as the environment increased in size, as every individual cell demanded careful attention. The lack of an intuitive interface forced developers to work directly with raw data, making modifications, debugging, and expansions extraordinarily time-consuming. The inherent complexity of this process discouraged experimentation and iteration, as even minor adjustments required significant effort.

Once an environment was finally completed, the next challenge arose. After processing the environment with Clingo, the solution would be presented as an unstructured and overwhelming block of action predicates. Parsing these predicates manually to understand the movement of trains was not only tedious but also highly inefficient. To address this issue, we initially implemented structured outputs, enabling a more organized representation of the generated actions. However, we soon realized that structuring the data alone was insufficient. A deeper level of clarity was necessary to effectively track and interpret railway operations.

Recognizing the limitations of text-based representations, we took the next logical step by developing a graphical visualization system. The introduction of numbered paths provided a breakthrough in understanding train routes, significantly reducing the effort required to analyze solutions. However, despite this improvement, our workflow remained constrained by technical limitations. The software we used in our course already offered notable features, including the ability to generate environments and create animations, but it also had serious shortcomings. During environment generation, we frequently encountered `OverflowErrors`, disrupting the workflow. Additionally, within the animation system, trains would occasionally disappear without any feedback, leaving users without an explanation for what had gone wrong. These limitations underscored the need for a more robust and feature-rich solution that could provide greater control, clarity, and stability.

Faced with these persistent challenges, we decided to take action. We conceived a comprehensive plan to address these issues and presented our vision for an integrated solution. Today, that vision has been fully realized in the form of Clingonia, a tool that has not only met our initial expectations but has also

evolved beyond them, incorporating additional features that enhance usability and efficiency.

Clingonia changes the way Flatland railway scheduling environments are built and managed. By bringing all essential functions into one unified interface, it removes the need for external tools and scattered workflows. Users can now generate, modify, save, load, and optimize environments seamlessly within one platform. The system provides multiple visualization modes, including numbered paths for immediate interpretation, detailed timetables for precision, and animated representations for dynamic insights into train movements.

One of the defining strengths of Clingonia is its simplicity. Where previous workflows required navigating through multiple folders, manually adjusting configurations, and dealing with scattered outputs, Clingonia streamlines the entire process. A single command brings all elements together, ensuring that users can focus on problem-solving.

With Clingonia, managing and solving Flatland environments is no longer a laborious and frustrating task but an intuitive and efficient experience. The journey that began with fragmented processes and technical constraints has now reached a new era of clarity, and control.

1.2 Structure

Clingonia’s repository is organized into distinct folders for ASP encodings, Python source code, data resources, and environment models, ensuring both flexibility and ease of use. Optimized for Windows and macOS, Clingonia also addresses its current limitations.

This report provides a structured overview of Clingonia’s development and findings. Section 2 presents a detailed breakdown of the individual Python source code files, explaining their modular implementation and functionality. Section 3 highlights the project’s key results through a collection of summarized visuals. Section 4 engages in a critical discussion, reflecting on innovative approaches, key insights, and achievements. Section 5 explores areas for improvement and potential directions for future research and development.

1.3 Technical Requirements

Clingonia is compatible with Python version 3.7 or higher, ensuring broad usability across contemporary Python environments. To ensure proper operation of Clingonia, the following Python modules must be installed: **flatland-rl** for creating and visualizing environments, **clingo** for the ASP solver used to compute train paths, **numpy** for efficient grid representation, **pandas** for managing train data, **matplotlib** for saving PNG files, **imageio** for generating GIFs, and **pillow** for image processing.

2 Implementation

2.1 main.py

This file is responsible for starting the program. It validates the prerequisites in several steps before initiating the main process.

The function `initial_import_authorization` checks the availability of modules by using `initial_import_test` from `files.py` to determine whether any modules are missing. If so, it invokes the error-handling routine of the `import_error_handling` function.

The `import_error_handling` function handles errors related to missing Python modules that are essential for the program to run. If this function is called during startup, it displays a corresponding message to the user and lists the necessary pip installation commands to install the missing modules. It then terminates the program in a controlled manner.

The function `initial_render_authorization` verifies the rendering functionality by using `initial_render_test` from `build_png.py` to detect whether an `OverflowError` occurs. If it does, the program is terminated in a controlled manner with an appropriate error message.

Once both checks have passed, the function `start_clingonia` is called to perform cleanup using `remove_data_remnants` from `files.py` and to properly launch the application using functions from `views.py`. If an error arises during startup, the program is also terminated in a controlled manner, accompanied by a suitable error message.

2.2 build_gif.py

This file is responsible for creating GIFs. Its imports form the basis for image processing and generation.

During the creation process, frames are collected in the `images` array. To avoid unnecessary re-rendering, the parameters are checked to see whether they have changed significantly since the last rendering, thus enabling an efficient caching strategy.

To render each individual frame, the environment is recreated and updated for every frame. The resolution is determined dynamically based on the environment's maximum dimension. The `RenderTool` then handles the actual rendering of each frame. Afterwards, the corresponding time step is drawn on every frame.

Finally, the frames are merged into an animated GIF at the desired frame rate and saved.

2.3 build_png.py

This file is responsible for creating, configuring, and visualizing environments. Several error-handling mechanisms are implemented here.

Custom environments are made possible through dummy classes with placeholders, enabling a minimal, runnable setup. The `RailEnv` object is based on a 2D list and agent information.

Agents with an invalid orientation on their track are corrected. In build mode, agent information is also validated, and if it is incorrect, the user is prompted to make adjustments.

Regarding rendering, parts of the PIL library are overridden to improve visualization. In very large environments, Low Quality Mode is automatically activated. The PNG resolution is determined dynamically based on the environment's maximum dimension, and the PNG image is stored temporarily after rendering.

For `main.py`, the `initial_render_test` is also defined here. It renders a simple 1x1 environment with a single horizontal track, one agent, and one station to test, at program start, whether the rendering works.

2.4 `clingo.actions.py`

This file is responsible for interacting with the ASP solver. Clingo is invoked as an external process.

Because Clingo itself does not provide intermediate status updates, humorous messages have been defined that, on the one hand, inform the user every 30 seconds (via a parallel thread) that Clingo is still running, and on the other hand, communicate the total runtime of Clingo. For this purpose, Clingo is launched with the ASP files as a separate process, limiting its output to the desired response. This output is stored, filtered for the user's requested response, and the action predicates are finally converted into a DataFrame.

Thanks to robust error handling that uses return codes, the code provides the user with detailed feedback.

2.5 `config.py`

This file is responsible for centrally managing important constants used across various files. These include the mapping of agent orientations, the Flatland track types, dead ends, and the agents' colors.

2.6 `custom.canvas.py`

`Custom_canvas.py` contains the environment displays that we created from Tkinter Canvases. If `views.py` is the brain of Clingonia's GUI, this is its heart. These canvases are what we use to display the created environments and to enable the user to modify them. The `EnvCanvas` is used in the Main menu and the generation and builder review frames to display the created environments. They provide a simple image of the environment together with a grid. The Canvas can be zoomed in and out as well as moved around to get a more detailed view of the user's creation. The `ResultCanvas` provides the additional functionality to display the paths of the calculated solution on the canvas, in such a way that the different train paths try to avoid overlapping their labels when possible and their colours match the ones of the train they belong to. The `BuilderCanvas` is the

main part of the build and modify section of Clingonia, it provides the modifiable grid where the user can place the track, train and station tiles. It automatically adds the trains and stations to the global train list and draws the assigned IDs onto the grid to allow the user to see which train belongs to which station. The canvas tries as with the path numbers to avoid overlapping if multiple trains or stations are within the same cell. The BuilderCanvas also provides the zoom and pan functionality of the Result- and EnvCanvas classes. The mentioned train list that the BuilderCanvas adds the trains to is also managed by the TrainListCanvas which provides the user with a quick overview of all the trains, their start position and their direction. it also holds a button for each train to modify the earlier departure and latest arrival and a button to remove the train from the environment. The PathListCanvas is very similar to the TrainListCanvas as it provides a list of all the trains with their ID together with a switch to display or hide this train path on the ResultCanvas. It also provides the function to toggle all paths on the ResultCanvas at once, to allow for a quick overview of all trains the user placed or to quickly clear the environment if it gets too cluttered. Both the TrainListCanvas and the PathListCanvas are scrollable lists to allow easy access to a large number of trains.

This module requires the user to have the PIL, numpy and pandas modules installed. As with views.py and custom_widgets.py PIL, the Python image library is used to handle the images of the environment. Numpy is used to handle the array that contains the environment which allows for quick accesses and modifications on large scales. The train list is stored as a pandas data frame which also allows for quick and easy modifications even with hundreds of trains. All other imports in this module are from the standard Python or Clingonia library.

2.7 custom_widgets.py

In custom_widgets.py, as the name suggests, we created customized versions of standard Tkinter widgets, to be used in views.py and in custom_canvas.py. This includes custom versions of the frame, label and text classes, which combine the creation of the widget using the desired stylization such as colour and size with the automatic placement at the desired position. The button, GIF, ZoomableGIF, EntryField and ToggleSwitch classes all need more complicated initialization parameter modifications such as the ToggleSwitch that has to be drawn on a canvas and then needs to be assigned the different actions it should perform when clicked. Similarly, ZoomableGIF also utilizes a canvas to provide the zoom and pan functionality for a GIF, whereas the static GIF class simply displays the GIF in a Tkinter label. The EntryField just has some additional function to provide an example text when the user has not entered a text, to provide a little help as to what inputs are expected. The only widget not based on the default Tkinter variant is the Button class that utilizes the ttk version for more customizability, like a different colour when the cursor hovers over it. The simplest modified version is the Window class which provides pre-made functions

to close or resize the window, for example, to switch from full screen to window mode.

As for `views.py` the only module not included in the standard Python or Clingonia library is the PIL module, which the user will need to install separately.

2.8 `files.py`

This file is responsible for storing and cleaning up data.

Two key functions here are `write_trains` and `write_tracks`, which, once triggered by `save_env`, write agent information from a DataFrame and the environment from a 2D list as properly formatted predicates into an `.lp` file.

For `main.py`, this file also defines the `initial_import_test`, which checks whether all the Python modules required by the application are available.

2.9 `gen_png.py`

This file is responsible for generating environments based on given parameters.

The `custom_sparse_line_generator` is a custom route generator for agents that defaults to a dummy generator if no specific instructions are provided. Agents always receive a time step of 1 as their earliest departure, while their latest arrival is determined by the size of the environment and the number of agents.

Because the standard initialization of the Rail environment with `env.reset()` in `create_env` can frequently fail due to faulty grid configurations, the `fallback_reset` function was introduced to make the generator more robust. Existing errors are corrected using `get_allowed_dirs` and `validate_track`.

Via `extract_tracks` and `extract_trains`, the environment is converted into a 2D list, and agent information is stored in a DataFrame to facilitate later visualization.

The `create_env` function generates the environment based on the provided parameters. In large, complex environments, Flatland might attempt to lay tracks beyond the environment's boundary. To address this, the `rail_gen_test` function attempts to create the environment at the desired dimensions; if it fails, it incrementally increases the size of the environment so that tracks can be placed, and then trims it back down to the desired dimensions. After successful initialization, it also ensures that all agents have valid positions.

Finally, the `gen_env` function brings everything together by extracting track data and agent information, then rendering and saving a PNG image. A time estimate is also provided to the user to help gauge the rendering duration.

2.10 `load_env.py`

This file is responsible for extracting and processing ASP-encoded environments.

The cell, train, start, and end predicates are extracted, validated, and sorted. Invalid track types are replaced with 0, and validation checks ensure the track layout is correct and consistent, and that every train has a start and end point. The implementation also allows for handling various types of errors.

The `load_env` function consolidates all this data by placing the environment into a 2D list and the agent information into a `DataFrame`.

2.11 positions.py

This file is responsible for calculating and validating agent positions on the grid.

The `position_df` function brings everything together. Using `clingo_to_df` from `clingo_actions.py`, actions are converted into a `DataFrame`, from which the position `DataFrame` is then derived.

The starting position is extracted from the agent information. Within the `next_pos` function, the three dictionaries `fw_tracks`, `to_left_tracks`, and `to_right_tracks` in `dir_change` help determine the agent's new orientation when changing positions, while `pos_change` calculates the agent's new coordinates based on that orientation.

The `build_df_pos` function evaluates Clingo's action predicates to identify subsequent positions and collects them in a `DataFrame`.

The `adjust_actions` function ensures that a valid action path can be created for each agent by checking whether the last position matches the coordinates of the target station. If it does not, the first action is assumed to be the starting position, then removed, and the process is retried. This continues until the respective agent reaches its station or runs out of actions. The `ensure_train_spawns` function guarantees that each agent has at least one starting position, so no agent remains without position information.

If at least one agent fails to reach its destination, even though Clingo reported a solution, `write_act_err.txt` logs the faulty train paths. Two text-based log files are generated: one in a reduced format containing just the action predicates, and another in a comprehensive format. This gives the user an opportunity to investigate further.

Because simulations may take longer, audio feedback was implemented to indicate the simulation's completion.

2.12 views.py

The `views.py` file contains all views used in Clingonia. We structured them by the menu or general order they appear in, starting with the program window and the start menu opened on startup. This block also includes the sub-view for the start menu help frame and the transitions to the other menus.

Next up is the main menu which also features a dedicated help page and a menu that appears when an environment is loaded, providing the user with a brief overview of what they have imported. This block also includes the Clingo parameter view, where the user can set the parameters for the Clingo solver. In this menu, the user can select the files containing their pathfinding algorithm that Clingo should utilise to solve the environments. From the start and main menu the user can switch to the generation view or the environment builder. The main menu additionally offers the modify menu, allowing users to alter a

previously created or loaded environment, as well as the Clingo parameter menu, which leads to the Results view. Let's go through them one by one starting with the generation menu. Here the user can set parameters that influence the size of the environment and the number of trains, as well as their behaviour. Once they are satisfied they can generate an environment and review the result in the generation environment viewer, where a zoomable and pannable version of the map is displayed along with a list of all the trains and their attributes such as starting and ending positions, starting direction, earliest departure and latest arrival. After reviewing they can return to the main menu. The next option is to build a new environment or modify an existing one. Both views function similarly with the small difference that the builder starts with an empty map and no trains, while the modify mode loads the existing environment along with its train list when entering the menu. Once the environment is loaded, the user can change the parameters of the environment before moving on to build new or edit existing track layouts until they are satisfied with the result. Then they can move on to place new trains, modify the existing schedule or remove trains and their stations entirely. At any point, the user can switch between the track and train menu to make any changes they desire. To make placing a large number of trains easier the user is provided with a menu to configure the earliest departure and latest arrival of all trains. All three menus include a help menu should the user not know what a specific button does or how a part of the program works. After they have completed creating their environment they can get a rendered image of it along with a list of the trains similar to the review menu of the generation section. Once the user has an environment they wish to solve, they can use the Clingo parameters page to make adjustments to the solver as specified in the main menu section and then let Clingo solve their environment. If there are no errors and Clingo finds a solution, the user will be directed to the results menu, where they can view a selection or view all paths of the trains, watch an animation of the solution or go through each time step one by one and view where the trains are in that moment. Should there be a problem with the actions produced by Clingo, a menu will be displayed to revive them. If the user is finished and returns to the main menu they can save their environments via the save button which also allows for a picture of the environment to be saved as well.

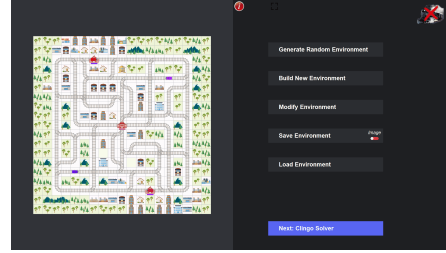
To use this module the user will need the third-party module PIL or "Python Image Library" or "Pillow" that is used to handle the environment images. In addition, the numpy and pandas modules will be required for handling the environment and train list respectively. Any other modules used are part of the standard Python or Clingonia library.

3 Project Results

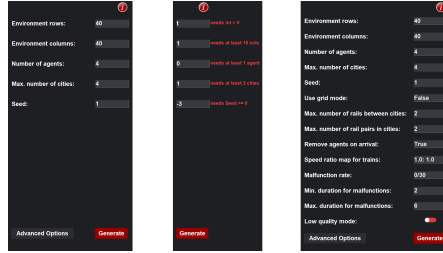
3.1 Menus & Generation



(a) Start Menu



(b) Main Menu



(c) Generation Parameters



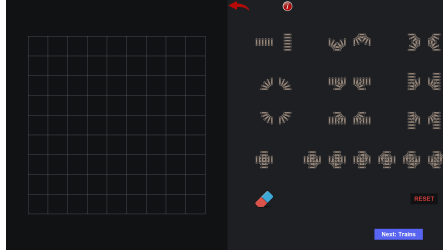
(d) Generation Overview

In the start menu (a), the user can choose from the three core functions of Clingonia: generation, build mode, and loading an environment.

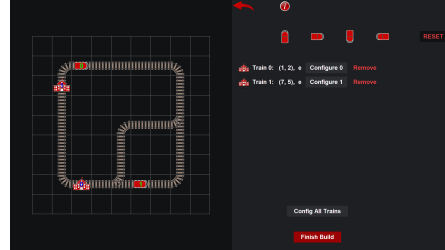
In the main menu (b), these options are presented again, now supplemented by the ability to edit and save the current environment, with or without an image. The user can also continue viewing the current environment here.

In generation mode (c), essential parameters are shown first and can be expanded if needed. Clingonia provides detailed feedback for invalid inputs. After clicking Generate, the environment is created and an image is rendered, which can then be viewed in an overview (d) alongside the parameters. The user can also hover over any cell in the environment to see its coordinates.

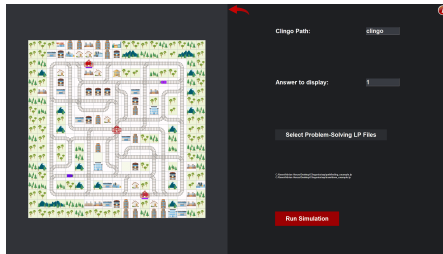
3.2 Build & Simulation



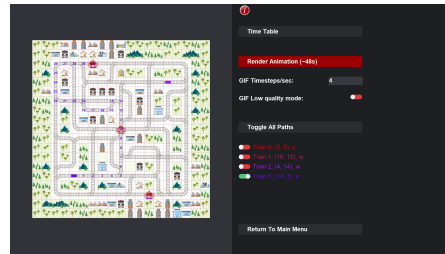
(a) Build: Railway Creation Lab



(b) Build: Train Configuration Lab



(c) Clingo Interface



(d) Results Page

After the user sets the grid size for Build mode, they enter the Railway Creation Lab (a). On the right side, various types of tracks are available, while on the left, the grid is ready for placing them. This mode is used both for creating new environments and modifying existing ones. The next step is the Train Configuration Lab (b), where trains and stations are placed on the newly created track system. Here, the earliest departure and latest arrival times for the trains can also be defined. After building is complete, the image of the environment is rendered.

In the Clingo Solver (c), the user can set up the solver by selecting the Clingo path and the corresponding .lp files. The Clingo command is launched, among other things, via the “Run Simulation” button, after which the Results Page appears.

On the Results Page (d), the outcome is presented in three different views: as a time table, as numbered train routes, or as a GIF animation. The playback speed of the GIF can be adjusted, and the animation can be viewed once rendering is finished. Additionally, users can click through individual frames manually and save the GIF.

4 Discussion

The plan we created prior to implementation proved invaluable for defining a clear structure for the program’s views. Initially, we attempted to use a single window by toggling the necessary elements on and off, but this approach fell short. We solved that problem quickly by using the current frame-based design, which was the only significant obstacle in developing the user interface.

Creating custom Tkinter widgets and using them exclusively proved very helpful. For example, when we switched from the standard Tkinter button to the ttk version, it automatically updated across all views without extra work. In addition, the number of required function calls decreased because they were already included in the initialization routines of the custom classes.

The environment generator was our biggest challenge. At first, agents failed to appear until we provided special hints. Even then, agents sometimes appeared misaligned, so we had to fix their placement. Initialization also sometimes failed, calling for robust error handling. Another issue arose from invalid track IDs and track placements beyond the environment’s boundary. We addressed this by temporarily expanding the environment’s range and then reverting it to its original dimensions.

Visualizing the train paths presented further difficulties, particularly when trains failed to reach their destinations or extra spawn actions showed up. To resolve these, we iteratively reduced the action sets until either a valid path existed or no actions remained. If a valid path could not be found, our “Action Error” Feature allows direct examination of the action predicates in Clingonia.

Another critical issue involved ensuring the stability of the RenderTool. Frequent OverflowErrors were frustrating at first and often stopped our finished work from rendering. We solved this by implementing an automatic render test at program startup. Careful balancing of image resolution was needed to preserve detail without consuming unnecessary resources in small or medium environments. Rendering was slow for large environments, so we added frame reuse and a low-quality mode to speed it up. While reviewing GIF frames, we noticed remnants of previous rendering attempts, which are now automatically removed under certain conditions.

We also focused on making everything as clear and intuitive as possible. To best assist users, we provide clear information through both the interface and the terminal. Repeated crashes caused by missing Python modules led us to implement an import test at startup. Older Python versions (prior to 3.10) also posed challenges, so we adapted Clingonia to be compatible. Since many students use macOS, we ensured macOS compatibility by making targeted adjustments and separating parts of the code.

We are especially proud of how we managed to generate valid paths for the final solution and integrate a build menu for creating custom environments. We hope these features will prove beneficial to future Flatland developers in their projects.

5 Future Research

We hope our program makes it easier for you to get started with Flatland and provides a smooth ride as you pass each station and achieve your goals. Even though Clingonia already tackles many challenges, there are still areas that could benefit from further research and development.

A key improvement would be to ensure Linux support, since so far we have only tested Windows and macOS. Another potential enhancement involves extending support if the course introduces a common fact format for speed ratios and malfunctions as ASP predicates. In that case, Clingonia would need to handle both converting such data into .lp files and reintegrating it into the program.

The current saving process can place a heavy load on RAM. A more efficient approach could lighten memory usage, and catching a potential `MemoryError` would help prevent crashes and increase stability. At present, dead ends are automatically replaced because they were not part of our course specifications; if this changes, adjustments to the implementation will be required. Validation in the build menu could also be enhanced so that the program itself checks whether the tracks lead to the desired destination rather than leaving it to the user.

Reducing the action predicates could benefit from further optimization. Right now, the process continues even after the shortest distance has been exceeded. A smarter limit would improve efficiency and reduce calculation time. We also welcome any further development, whether fixing errors, adding new features, or continuously adapting Clingonia to evolving Flatland requirements to keep it up to date.

Although Clingonia is not perfect, it lays the groundwork for an efficient, visual ASP-based solution in the Flatland environment. We hope this lowers the entry barrier and motivates you to create your own ideas to make Flatland even better.

Best Wishes,
MadMotion